

FinStructBench:

A Benchmark for Structured Information Retrieval
from Financial Documents Using Graph-Verifiable Questions

Agus Sudjianto*

Wingyan Lau†

Abstract

We introduce **FinStructBench**, a benchmark for evaluating the ability of large language models (LLMs) to extract, aggregate and reason over structured information in financial documents. Unlike existing QA benchmarks that rely on human-annotated answers, FinStructBench generates questions *automatically* from a document’s underlying knowledge graph, making ground truth *provably correct by construction*. We define ten question categories of increasing structural complexity—exact recall, threshold checking, cross-reference, counting, contradiction detection, multi-hop reasoning, absence detection, ranking, numeric computation and cross-table aggregation—and provide five benchmark instances spanning major financial report types: model validation (SR 11-7), fair lending (ECOA/HMDA), stress testing (CCAR/DFAST), credit portfolio review (OCC) and Basel III capital adequacy (Pillar 3). The benchmark supports two question modes: *deterministic* (template-based) and *hybrid* (LLM-paraphrased for linguistic diversity). Across 454 questions evaluated in both modes, the graph-based retrieval baseline achieves 100% accuracy by design, while Claude Sonnet 4 scores 58% (deterministic) and 59% (hybrid), with per-category accuracy ranging from 94% on absence detection to 30% on contradiction. A three-way ingestion comparison (regex-only, LLM-only and hybrid) using Claude Opus 4 demonstrates that LLM-only extraction recovers only 12.5% of structured data, while hybrid ingestion preserves 100% of regex values and adds 22% more entries from prose—empirically validating the regex-first, LLM-augmented architecture. FinStructBench is released as an open-source toolkit enabling researchers to generate benchmark instances from any markdown-formatted financial document.

1 Introduction

Financial regulation demands precise information extraction from structured documents: capital ratios must be reported to basis-point accuracy, adverse impact ratios must be computed across protected classes and stress-test projections must be traced across scenarios and time horizons. Errors in these tasks carry material consequences—regulatory findings, enforcement actions and capital shortfalls.

Large language models are increasingly deployed for financial document analysis, yet existing benchmarks for financial NLP focus predominantly on sentiment analysis [Malo et al., 2014], named entity recognition [Alvarado et al., 2015] or free-form question answering [Chen et al., 2021, Zhu et al., 2021] where ground truth is established by human annotation. These benchmarks have three limitations in the structured-retrieval setting:

1. **Annotation cost and error.** Human annotators make mistakes on precise numeric lookups, especially in dense tables with hundreds of cells.

*H2O.ai and Center for Trustworthy AI Through Model Risk Management, University of North Carolina Charlotte

†Knowlytix.ai

2. **Limited coverage.** Hand-crafted questions test only the patterns the annotator anticipated, missing systematic failure modes.
3. **No structural baseline.** Without a provably correct retrieval system, it is impossible to distinguish LLM errors from ambiguous ground truth.

We address all three limitations with **FinStructBench**, a benchmark construction methodology that:

- Ingests any markdown-formatted financial document into a *DocumentGraph* comprising exact numeric memory (ENM) entries and knowledge graph (KG) triples;
- Automatically generates questions by mining graph topology, with ground truth defined by graph traversal—making it provably correct;
- Provides a *graph-based retrieval baseline* (the graph itself) that achieves 100% by construction, against which any LLM or agentic system can be compared.

Our initial release includes 454 questions across five financial report types and ten complexity categories. Results with Claude Sonnet 4 reveal a 41–42 percentage-point gap between graph-based retrieval (100%) and LLM performance (58–59%), with per-category accuracy ranging from 94% on absence detection to 30% on contradiction. A three-way ingestion comparison further demonstrates that LLM-only extraction recovers only 12.5% of structured data, while a hybrid regex+LLM pipeline preserves 100% of deterministic values and adds 22% more entries from prose extraction. These findings demonstrate that current LLMs lack the structured traversal capabilities required for reliable financial document analysis and that deterministic ingestion is a quantitative necessity, not merely a design preference.

2 Related Work

Financial NLP Benchmarks. FinQA [Chen et al., 2021] and TAT-QA [Zhu et al., 2021] evaluate numerical reasoning over financial tables, but rely on human-annotated answers and focus on arithmetic operations (addition, percentage change) rather than structural graph operations. ConvFinQA [Chen et al., 2022] extends to conversational settings but inherits the same annotation limitations. FLUE [Shah et al., 2022] aggregates multiple financial NLP tasks but does not address structured retrieval from regulatory documents.

Table QA and Table-Specialized Models. WikiTableQuestions [Pasupat & Liang, 2015] and HybridQA [Chen et al., 2020] benchmark table understanding but use general-domain tables without the precision requirements of financial regulation. SQA [Iyyer et al., 2017] addresses sequential table queries but does not test cross-table reasoning. TAPAS [Herzig et al., 2020] extends BERT to encode tables jointly with questions, selecting cells and applying aggregation operators. TaPEX [Liu et al., 2022] pre-trains a generative model as a neural SQL executor, achieving strong results on WikiTableQuestions. However, both are limited to single-table contexts and struggle with the multi-step numerical reasoning required in financial analysis.

Document Understanding. DocVQA [Mathew et al., 2021] and financial variants evaluate visual document understanding, which is complementary to our text-based setting. Layout-aware

models such as LayoutLMv3 [Huang et al., 2022] jointly encode text, layout, and visual features, while Donut [Kim et al., 2022] performs OCR-free document understanding end-to-end and Pix2Struct [Lee et al., 2023] learns to parse screenshots into structured markup. These models address the upstream task of extracting clean tables from document images, which is complementary to FinStructBench’s focus on reasoning *over* already-extracted structured data. Recent work on long-context evaluation [Li et al., 2024] tests retrieval from long documents but uses synthetic “needle-in-haystack” probes rather than domain-specific structural queries.

Structured Reasoning and Tool Use. Chain-of-Table [Wang et al., 2024] guides LLMs to iteratively transform tables through atomic operations (select, group, sort) as intermediate reasoning states, achieving strong results on table QA. DATER [Ye et al., 2023] decomposes both tables and questions into simpler sub-problems for LLM reasoning. StructGPT [Jiang et al., 2023] constructs specialized interfaces to collect evidence from structured data (tables, knowledge graphs, databases) and lets LLMs focus on reasoning over the collected information. ReAct [Yao et al., 2023] interleaves reasoning traces with tool-calling actions, and Toolformer [Schick et al., 2023] trains models to self-select when to invoke external tools such as calculators and search APIs. These tool-augmented approaches are directly relevant to FinStructBench’s MCP server architecture, where the LLM calls deterministic graph operations rather than extracting structured data from raw text.

Graph-Augmented Retrieval. GraphRAG [Edge et al., 2024] builds entity knowledge graphs from source documents and pre-generates community summaries for query-focused retrieval, outperforming vector-based RAG on global sensemaking tasks. KGQA benchmarks [Zhang et al., 2022] evaluate question answering over pre-existing knowledge graphs. FinStructBench differs from both in that the graph is *constructed from* the document using purely deterministic rules, and the benchmark tests whether an LLM can implicitly perform the same traversal from the raw text without access to the graph.

Question Generation Methodologies. Automatic question generation (QG) from structured data spans a broad spectrum of approaches. At one extreme, classical benchmarks such as FinQA [Chen et al., 2021], TAT-QA [Zhu et al., 2021], and FinanceBench [Islam et al., 2023] rely entirely on human expert annotation—a process that is expensive (\$2+ per question), inherently noisy, and difficult to scale. At the other extreme, fully LLM-driven approaches have gained traction: FinForge [Matlin et al., 2026] uses a five-stage pipeline with Gemini 2.5 Flash to generate semi-synthetic financial QA pairs, YourBench [Shashidhar et al., 2025] constructs zero-shot evaluation sets from arbitrary user documents, and BenchAgents [Butt et al., 2024] decomposes benchmark creation into multi-agent planning, generation, verification, and evaluation stages. While these LLM-based methods achieve high linguistic diversity, they introduce non-determinism, contamination risk (the generating model may have memorized benchmark data), and dependence on API availability.

Between these extremes, *programmatic graph-topology mining* generates questions by algorithmically traversing a knowledge graph and rendering the resulting logical forms as natural language. GrailQA [Gu et al., 2021] enumerates SPARQL queries over Freebase by traversing graph structure up to four relation hops, then crowdsources natural-language paraphrases. LC-QuAD [Trivedi et al., 2017] follows a similar template-based SPARQL generation pipeline over DBpedia. In the financial domain, KG-QAGen [Tatarinov et al., 2025] generates 20,139 QA pairs from structured credit agreements by systematically varying complexity along multi-hop retrieval depth, set operations, and answer plurality. Dynamic-KGQA [Dammu et al., 2025] extracts semantically coherent subgraphs

from YAGO 4.5 and uses an LLM to generate questions, producing fresh train/test splits each run to resist data contamination. More recently, several benchmarks have formalized table-level reasoning complexity: TableBench [Wu et al., 2025] defines 18 sub-categories of table reasoning, TableEval [Zhu et al., 2025] combines template-prompted and role-prompted LLM strategies with clustering and human verification, and GraphArena [Tang et al., 2025] benchmarks LLMs on graph computational problems (shortest path, connectivity) over diverse graph topologies.

FinStructBench occupies a distinctive position within this landscape. Like GrailQA and KG-QAGen, it mines graph topology programmatically to generate questions with provably correct ground truth. However, it differs in three key respects: (i) the knowledge graph is *constructed from* the document via deterministic ingestion rather than being a pre-existing knowledge base; (ii) it defines ten specialized generator categories tailored to financial analyst workflows rather than general SPARQL templates; and (iii) no LLM or crowdsourcing is involved at any stage of question creation, ensuring perfect reproducibility, zero contamination risk, and zero marginal generation cost. The tradeoff is limited linguistic diversity in the generated questions—a limitation we address with an optional hybrid mode that uses LLM paraphrasing while preserving the deterministic ground-truth guarantee (Section 3.2).

3 Benchmark Construction

FinStructBench’s construction pipeline has three stages: document ingestion, question generation, and evaluation. Each stage is fully automated and domain-agnostic. Critically, the entire pipeline is *purely deterministic*—it relies exclusively on regex pattern matching, string parsing, and rule-based heuristics, with no machine learning or LLM involved at any point. This design is deliberate: because the ground-truth graph is constructed by verifiable, transparent rules rather than by a model that could hallucinate or make probabilistic errors, the resulting answers are provably correct by construction.

3.1 Document Ingestion

Given a markdown-formatted financial document, the ingester produces a **DocumentGraph** $\mathcal{G} = (\mathcal{E}, \mathcal{T}, \Phi)$ with three components:

Exact Numeric Memory (ENM) $\mathcal{E}$: A key-value store mapping $(type, id) \rightarrow (value, hash)$ where each numeric value is stored with its SHA-256 hash for integrity verification. The *type* is derived from the sanitized section heading (e.g., `capital_adequacy_ratios`), and the *id* is a composite key encoding the entity, any secondary label columns, and (for multi-column tables) the column name, separated by `/`. For example, a stress test table with columns Scenario, Quarter, CET1, and Tier 1 produces keys such as `(capital_projections, Baseline/Q1 2026/CET1)`, ensuring unambiguous lookup even when the same metric name appears in multiple rows. Each value is stored alongside its SHA-256 hash (computed over the IEEE-754 double representation) so that any silent corruption is detected at read time.

Knowledge Graph Triples $\mathcal{T}$: A set of $(head, relation, tail)$ triples extracted from table structure. For each row the ingester emits: (i) a `has_col` triple for every numeric cell, linking the entity to its string-formatted value; (ii) a categorical triple for every non-entity label column; (iii) a `passes` or `fails` triple if a boolean column is present; and (iv) an `in_section` triple linking the entity to its section tag.

Algorithm 1 Document Ingestion Pipeline

Require: Markdown document D

Ensure: DocumentGraph $\mathcal{G} = (\mathcal{E}, \mathcal{T}, \Phi)$

```
1: Parse  $D$  into sections  $\{s_1, \dots, s_k\}$  by ##/### heading hierarchy
2: Infer phase encoders  $\Phi$  from column names across all tables
3: for each section  $s_i$  do
4:   for each markdown table  $T_j$  in  $s_i$  do
5:     Classify columns into entity, numeric, boolean, label
6:     Detect entity column by name heuristic; detect boolean column
7:     for each row  $r$  in  $T_j$  do
8:        $e \leftarrow$  entity value (or subsection name, or row- $i$ )
9:        $e_{\text{full}} \leftarrow e / \text{label}_1 / \dots / \text{label}_m$  {composite ID}
10:      for each numeric column  $c$  do
11:         $id \leftarrow e_{\text{full}}/c$  if multi-column, else  $e_{\text{full}}$ 
12:         $\mathcal{E}[\text{section\_tag}, id] \leftarrow (v_c, \text{SHA-256}(v_c))$ 
13:         $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e, \text{has\_}c, v_c)\}$ 
14:      end for
15:      Add label triples:  $(e, \text{has\_}\ell, v_\ell)$  for each label column  $\ell$ 
16:      if boolean column value  $\in \{\text{true}, \text{yes}, \text{weak}, \text{fail}\}$  then
17:         $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e_{\text{full}}, \text{fails}, \text{tag})\}$ 
18:      else if  $\in \{\text{false}, \text{no}, \text{strong}, \text{pass}\}$  then
19:         $\mathcal{T} \leftarrow \mathcal{T} \cup \{(e_{\text{full}}, \text{passes}, \text{tag})\}$ 
20:      end if
21:    end for
22:  end for
23:  Extract bullet-list key-value pairs as metadata ENM entries and triples
24: end for
25: return  $\mathcal{G}$ 
```

Phase Encoders Φ : Inequality functions that map a metric type to a comparison operation.

Each encoder stores the value range $[v_{\min}, v_{\max}]$ observed for that metric (used internally for geometric encoding) and exposes a `check_inequality( $v, \tau, \text{op}$ )` method supporting $\geq$, $>$, $\leq$, and $<$. Encoders are inferred from column names using regex matching against a dictionary of 11 financial metric types (Table 1).

Step-by-Step Ground-Truth Construction. We now describe the complete process by which a markdown document is transformed into a provably correct ground-truth graph, using a concrete example. Consider the following table appearing under the heading **## Capital Adequacy Ratios**:

Metric	Value	Status
CET1	12.5	Pass
Tier_1	14.3	Pass
Total	18.1	Fail

The ingester processes this table through five sequential steps:

1. **Section Parsing.** The document is split into hierarchical sections by `##` and `###` headings. The heading “Capital Adequacy Ratios” is sanitized into the section tag `capital_adequacy_ratios` by stripping leading numbering (e.g., “3.1”), removing non-alphanumeric characters (except underscores and spaces), replacing spaces with underscores, and lowercasing.
2. **Phase Encoder Registration.** Before processing any tables, the ingester concatenates the full text of all sections and scans it against 11 regex patterns (Table 1). Matching is performed on the entire section content—including entity names, column headers, and prose text—not just column names. In this example, the entity names “CET1” and “Tier_1” match the pattern `(?i)\b(CET1|Tier.?1|capital_ratio|leverage)\b`, registering the `capital_ratio` encoder with value range $[0, 30]$. The encoder itself stores only the range $(v_{\min}, v_{\max})$; the threshold values used for question generation (e.g., ≥ 4.5 for capital ratios) are defined separately in the Threshold question generator. Encoders are registered once per metric type and shared across all sections.
3. **Column Classification.** Each column’s values are analyzed to determine its type. In this example: “Metric” contains strings $\rightarrow$ *label*; “Value” contains numbers (100% parseable as float) $\rightarrow$ *numeric*; “Status” contains exclusively pass/fail keywords $\rightarrow$ *boolean*. The “Metric” column is further identified as the *entity* column because it matches the name heuristic (a list of 16 common column names including “Metric”).
4. **ENM Entry Construction.** For each row, the entity name and numeric value(s) are extracted. Since this table has a single numeric column, the ENM key is the entity name directly (no column suffix needed). The ingester stores:

$$\begin{aligned}\mathcal{E}[\text{capital_adequacy_ratios}, \text{CET1}] &= (12.5, \text{SHA-256}(12.5)) \\ \mathcal{E}[\text{capital_adequacy_ratios}, \text{Tier_1}] &= (14.3, \text{SHA-256}(14.3)) \\ \mathcal{E}[\text{capital_adequacy_ratios}, \text{Total}] &= (18.1, \text{SHA-256}(18.1))\end{aligned}$$

Each SHA-256 hash is computed over the IEEE-754 double-precision representation and verified on every subsequent lookup, ensuring bit-exact integrity throughout the benchmark life-cycle.

For tables with multiple numeric columns (e.g., columns AUC, Precision, Recall), the column name is appended to form a composite key: `(type, Entity/Column)`. Tables with secondary label columns (e.g., Scenario and Quarter) produce keys such as `Baseline/Q1 2026/CET1`.

5. **Triple Extraction.** The ingester emits knowledge graph triples that capture the full relational structure of each row. Numeric and section triples use the base entity name as head; boolean triples use the full composite entity ID (which equals the base name when no secondary labels are present):
 - *Numeric triples:* For each numeric cell, a `has_col` triple is emitted using the base entity name: `(CET1, has_value, 12.5)`, `(Tier_1, has_value, 14.3)`, `(Total, has_value, 18.1)`.
 - *Boolean triples:* The “Status” column is boolean. Values `pass`, `false`, `no`, and `strong` map to a `passes` relation paired with an `is_strong` triple. Values `fail`, `true`, `yes`, and `weak` map to `fails` paired with `is_weak`. In this example: `(CET1, passes, capital_adequacy_ratios_CET1)`, `(CET1, is_strong, capital_adequacy_ratios)`; `(Total, fails, capital_adequacy_ratios_Total)`, `(Total, is_weak, capital_adequacy_ratios)`.
 - *Section triples:* `(CET1, in_section, capital_adequacy_ratios)`, etc.

Table 1: Phase encoder types: column-name patterns, default question thresholds, and comparison direction. Thresholds are domain-standard regulatory or statistical bounds; the Direction column shows whether values should be above ($\geq$) or below ($\leq$) the threshold.

Metric Type	Pattern	Threshold	Direction
AUC	auc	0.5	$\geq$
Accuracy	acc, accuracy	0.5	$\geq$
AIR	air	0.8	$\geq$
Ratio	ratio	0.0	$\geq$
Rate	rate, default.rate, approval.rate	0.05	$<$
Loss	loss, logloss, brier	0.5	$<$
Capital Ratio	cet1, tier1, capital.ratio, leverage	4.5	$\geq$
Importance	importance, score, weight	0.1	$\geq$
Divergence	divergence, distance, js, kl	0.3	$\geq$
P-value	p.value, pvalue	0.05	$<$
Percentage	pct, percent, %	0.0	$\geq$

After all sections are processed, the resulting DocumentGraph is self-contained: every numeric value in the source document has a corresponding ENM entry (verifiable by text search), every relational fact is encoded as a triple, and every applicable threshold is registered as a phase encoder. This completeness guarantee is what makes the graph a *provably correct* ground truth—any question generated from this graph can be answered by deterministic traversal, and the answer can be independently verified against the source text.

Column Classification. The ingester classifies each column into one of four types using value-pattern analysis:

- **Numeric:** columns where $>70\%$ of non-empty values parse as floats (after stripping \$, %, commas, and parentheses for negative values).
- **Boolean:** columns where *every* non-empty value is one of: pass, fail, yes, no, true, false, weak, strong (case-insensitive).
- **Label:** non-numeric, non-boolean string columns.
- **Entity:** the primary identifier column, selected by name heuristic from a list of 16 common column names (e.g., “Feature”, “Segment”, “Scenario”, “Product”), falling back to the first label column.

This classification is conservative by design: mixed columns (e.g., 3.2 (Pass)) are classified as label rather than boolean or numeric, meaning the corresponding pass/fail relationship is not captured in the graph. Questions are generated only from data that *is* captured, so coverage may be incomplete but ground truth is never wrong.

Phase Encoders. Phase encoders serve two roles: (i) they define the value range for normalizing numeric values to $[0, 1]$, and (ii) they provide the threshold values and comparison operators used by the Threshold question generator. Table 1 lists the 11 encoder types, their column-name patterns, and the default threshold used for question generation (not the encoding range).

3.2 Question Generation

Questions are generated automatically by mining the DocumentGraph topology. Each generator implements a `generate(graph)` method that returns a pool of candidate questions, each comprising: (i) a natural-language question string, (ii) a deterministic graph-traversal function that computes the ground-truth answer, (iii) a structured LLM prompt with format instructions, and (iv) a typed answer format (float, int, bool, set, or string). We define ten categories of increasing structural complexity. Below we describe the precise mining algorithm for each category, including the graph traversal patterns, filtering heuristics, and sub-patterns that determine which questions are generated from a given document.

3.2.1 Category 1: Exact Recall

Purpose. Test whether an LLM can look up a single numeric value by its structured key—the most fundamental retrieval operation.

Mining algorithm. The generator iterates over every ENM entry $(type, id) \rightarrow (value, hash)$ in the graph. Each composite ENM ID is parsed into a base entity name and an optional column name by splitting on `/`: `Entity/Label1/.../Column` yields (entity = `Entity`, column = `Column`). Entries whose entity name matches the synthetic pattern `row_\d+` (generated when no entity column was detected) are skipped, since these names do not appear in the source document and would produce unanswerable questions.

For each surviving entry, the generator produces one question. If a column name is present, the question specifies both the entity and the column (e.g., “*What is the ‘AUC’ value for ‘Model_A’ (category: model_validation)?*”). If the ENM ID has no column component (single numeric column in the source table), the question asks for the entity’s value directly.

Graph answer function. $\mathcal{E}[type, id].value$ — a single key lookup with SHA-256 integrity verification.

Answer type. float. The LLM prompt instructs exact reproduction of all decimal places.

3.2.2 Category 2: Threshold

Purpose. Test whether an LLM can locate a value and compare it against a domain-specific regulatory or statistical bound.

Mining algorithm. For each ENM entry, the generator attempts to match it to one of 11 phase encoder types using a three-stage process:

1. **Word-boundary regex matching.** The concatenation of the entry’s category and entity ID is tested against a list of compiled regex patterns with word boundaries (e.g., `\bauc\b`, `\bcet1\b`, `\bcapital.ratio\b`). Word boundaries prevent false positives such as “pair” matching the AIR encoder.
2. **Semantic exclusion.** For the `capital_ratio` encoder specifically, entities matching dollar-denominated field names (e.g., `dividend`, `provision`, `charge_off`, `rwa`, `goodwill`) are excluded even if they appear in a capital-related category, because these are dollar amounts rather than ratios.

3. **Value plausibility.** The entry’s numeric value is checked against the encoder’s threshold range. Entries with $|\text{value}| > \max(\tau \times 100, 1000)$ (where τ is the largest threshold for that encoder) are rejected, filtering out dollar amounts that passed the regex match.

If all three checks pass and the matched encoder is registered in the graph, the generator produces one boolean question for each threshold defined in Table 1 (e.g., $\text{AUC} \geq 0.5$, $\text{AIR} \geq 0.80$, capital ratio $\geq 4.5\%$).

Graph answer function. $v \leftarrow \mathcal{E}[\text{type}, \text{id}]$; then $\phi_{\text{encoder}}.\text{check_inequality}(v, \tau, \text{op}) \rightarrow \text{bool}$.

Answer type. `bool`.

3.2.3 Category 3: Cross-Reference

Purpose. Test set intersection across two relation types, requiring the LLM to identify entities appearing in two distinct relational contexts.

Mining algorithm. The generator builds an inverted index mapping each relation r to its set of head entities: $H_r = \{h : (h, r, -) \in \mathcal{T}\}$. High-cardinality, low-information relations (`in_section`, `has_value`, `has_effect`) are excluded. Relations with fewer than 2 head entities are also excluded.

For each pair of surviving relations (r_1, r_2) , the generator computes $H_{r_1} \cap H_{r_2}$. Only pairs with $2 \leq |H_{r_1} \cap H_{r_2}| \leq 50$ are retained; intersections below 2 are trivial, and those above 50 are too noisy for meaningful evaluation.

Graph answer function. $\{h : (h, r_1, -) \in \mathcal{T}\} \cap \{h : (h, r_2, -) \in \mathcal{T}\} \rightarrow \text{set of entity name strings}$.

Answer type. `set_str` (comma-separated list).

3.2.4 Category 4: Counting

Purpose. Test exhaustive enumeration and aggregation over triple sets—a task where LLMs tend to undercount by stopping before scanning all triples.

Mining algorithm. The generator produces four sub-patterns of counting questions:

- (a) **Count by relation.** For each relation r (excluding noisy relations), count the total number of triples: $|\{(h, r, t) \in \mathcal{T}\}|$. Relations with fewer than 2 triples are skipped.
- (b) **Count by (relation, tail).** Group triples by (r, t) pairs and count unique head entities per group: $|\{h : (h, r, t) \in \mathcal{T}\}|$. Only groups with 2–50 heads are retained.
- (c) **Count unique heads.** For each relation r , count the number of distinct entities that participate: $|\{h : \exists t, (h, r, t) \in \mathcal{T}\}|$. Requires ≥ 3 unique heads.
- (d) **Conditional count (above median).** A multi-step pattern that combines triple queries with ENM aggregation. For each (r, t) group with ≥ 3 head entities, and for each ENM column where at least 3 of those entities appear: (i) compute the median of all values in that column (not just the overlapping entities); (ii) count how many entities from the (r, t) group have a column value strictly above the median. Questions where the count equals zero or equals the full overlap size (trivial answers) are discarded.

Graph answer function. Varies by sub-pattern: (a–c) use `query_triples` with pattern matching and `len()`; (d) chains `query_triples` → ENM iteration → `median` → filtered count.

Answer type. `int`.

3.2.5 Category 5: Contradiction

Purpose. Test global consistency checking—whether the LLM can detect that a feature passes validation in some segments but fails in others.

Mining algorithm. The generator calls `graph.find_contradictions()`, which identifies entities possessing both `passes` and `fails` triples within the same feature group. Feature groups are determined by splitting the test-name string on `_` (first component) and then on `:` and `/` (to extract the feature name from composite test identifiers). Only entities where the passing and failing tests share the same group prefix are considered contradictions—this avoids flagging entities that legitimately pass one test and fail a different, unrelated test.

If contradictions exist, three sub-types of questions are generated:

- (a) **Count:** How many features have contradictory performance? (`int`)
- (b) **Set:** Which features have contradictions? (`set_str`)
- (c) **Per-feature boolean:** For each feature that participates in any pass/fail triple (not just contradicting ones), ask whether it has contradictory performance. This produces both true-expected and false-expected questions. (`bool`)

This category is only applicable to instances with boolean columns; in our release, only the model validation instance generates contradiction questions.

Graph answer function. Calls `find_contradictions()` and extracts unique feature names via prefix parsing.

Answer type. `int`, `set_str`, or `bool`.

3.2.6 Category 6: Multi-Hop

Purpose. Test chained multi-step reasoning that requires maintaining intermediate results across sequential graph operations—the most structurally complex category.

Mining algorithm. ENM entries are grouped by type, then by column within each type using the `/`-separated structure of composite ENM IDs. Columns with fewer than 3 non-aggregate entries are excluded (aggregate/summary rows such as “Total”, “Grand Total”, “Overall” are detected by regex and skipped to avoid trivially dominant extrema). The generator produces five sub-patterns:

- (a) **Argmax/min → cross-category lookup (float).** For each column c_1 in type t_1 , find the entity with the highest (or lowest) value. Then search for that same entity (matched by base name) in a different type t_2 , column c_2 . A question is generated only if the bridge entity exists in both types. The answer is the numeric value from the second lookup.
“Which entity has the highest ‘AUC’ in ‘model validation’? What is that entity’s ‘AIR’ in ‘fair lending’?”

- (b) **Spread / range (float)**. For each column within a single type, compute $\max(c) - \min(c)$ across all non-aggregate entities. Columns where the spread is zero (all values identical) are skipped.
“What is the range (highest minus lowest) of ‘AUC’ values in ‘model validation’?”
- (c) **Argmax/min entity name (str)**. For each column, return the name of the entity with the highest or lowest value. Columns with tied extrema are skipped to ensure an unambiguous answer.
“Which entity has the lowest ‘Loss_Rate’ in ‘credit portfolio’?”
- (d) **Conditional filter $\rightarrow$ count (int)**. Chains a KG triple query with ENM threshold checking. For each (r, t) group with ≥ 3 head entities, and for each ENM column where those entities appear: match the column to a phase encoder (using the same three-stage matching as Threshold), then count how many entities from the triple group meet the encoder’s threshold. Requires at least 2 overlapping entities.
“How many entities that ‘passes’ ‘test_X’ also have ‘AUC’ ≥ 0.5 in ‘model validation’?”
- (e) **Argmax/min $\rightarrow$ cross-category threshold check (bool)**. A three-step chain: (i) find the entity with the highest/lowest value in column c_1 of type t_1 (skipping ties); (ii) look up that entity’s value for column c_2 in type t_2 ; (iii) check whether the cross-looked-up value meets a phase encoder threshold.
“Does the entity with the highest ‘AUC’ in ‘model validation’ meet the 4/5ths rule for ‘AIR’ in ‘fair lending’?”

Graph answer function. Each sub-pattern iterates over the ENM, maintaining intermediate state (the identity of the extremal entity, the cross-looked-up value) before returning the final answer.

Answer type. float, str, int, or bool depending on sub-pattern.

3.2.7 Category 7: Absence

Purpose. Test hallucination resistance—whether the LLM can correctly verify that an entity does *not* appear in a given section, rather than fabricating a plausible value.

Mining algorithm. The generator collects the set of base entity names per ENM type (excluding synthetic row_N and aggregate names). It requires at least two distinct types to generate questions. Three sub-patterns of boolean questions are produced:

- (a) **Cross-type absent (false-expected)**. For each target type t , find entities that exist in some other type t' but not in t . These are plausible-sounding names (they exist somewhere in the document) that the LLM might hallucinate a value for in the wrong section.
- (b) **Near-miss distractors (false-expected)**. For each entity present in type t , generate string variants that do not exist:
- *Prefix variants:* Progressively truncate the entity name (e.g., **Segment_A1** $\rightarrow$ **Segment_A**). The prefix must be at least 3 characters long, must not itself be present in the target type, and must exist as a real entity in some other type (ensuring it is a plausible name, not random truncation).

- *Suffix variants*: Append one of three suffixes (0, a, _v2) to a present entity name (e.g., Model_1 $\rightarrow$ Model_1a). The variant must not exist anywhere in the graph.
- (c) **Positive controls (true-expected)**. For each entity that *is* present in type t , generate a confirmation question. This balances the question pool to prevent a trivial “always answer false” strategy.

Graph answer function. Scans all ENM keys of the target type, parsing each to its base entity name, and returns **True** if a match is found, **False** otherwise.

Answer type. bool.

3.2.8 Category 8: Ranking

Purpose. Test whether the LLM can extract all values in a column, sort them correctly, and identify the entity at a specific ordinal position—stressing both exhaustive scanning and ordering accuracy.

Mining algorithm. ENM entries are grouped by (type, column). Groups with fewer than 3 non-aggregate entries or with any tied values are discarded entirely (ties would make rank positions ambiguous). Within each surviving group, entries are sorted in descending order (rank 1 = highest). Questions are generated for a fixed set of key positions:

- Rank 1 (highest) and rank n (lowest), always included.
- Rank 2 (second highest), if $n \geq 3$.
- Median position ($\lfloor n/2 \rfloor + 1$), if $n \geq 5$.

Graph answer function. Collects all (entity, value) pairs for the target (type, column), sorts descending, returns the entity name at position k .

Answer type. str (entity name).

3.2.9 Category 9: Numeric Computation

Purpose. Test multi-step numeric reasoning: extracting two or more values and performing arithmetic—a key failure mode identified in recent work on table QA.

Mining algorithm. ENM entries are grouped by (type, column). Two sub-patterns are generated:

- (a) **Pairwise arithmetic (float)**. For every pair of entities (e_a, e_b) sharing the same column (generated via $\binom{n}{2}$ combinations), four operations are applied:
- *Sum*: $v_a + v_b$
 - *Difference*: $v_a - v_b$
 - *Ratio*: v_a/v_b (skipped if $|v_b| < 10^{-12}$)
 - *Percentage change*: $\frac{v_b - v_a}{|v_a|} \times 100$ (skipped if $|v_a| < 10^{-12}$)

Requires ≥ 2 entities in the column.

(b) **N-ary aggregate (float)**. For each column with ≥ 3 entities, compute:

- *Mean*: arithmetic average of all values in the column.
- *Median*: middle value (or average of two middle values).

The LLM prompt explicitly states the number of entities to extract, testing completeness of value extraction.

Graph answer function. (a) Two `lookup` calls followed by the arithmetic operation; (b) iteration over all ENM entries matching the (type, column) pair, then `mean` or `median`.

Answer type. float.

3.2.10 Category 10: Cross-Table Aggregation

Purpose. Test whether the LLM can locate the same entity across multiple table sections and perform arithmetic on the corresponding values—a common task in financial analysis where metrics are reported in different contexts (e.g., baseline vs. stress scenarios).

Mining algorithm. The generator builds an index mapping each base entity name to all its ENM occurrences: $\text{entity} \rightarrow [(t_i, c_i, \text{eid}_i, v_i), \dots]$. Entities appearing in fewer than 2 distinct ENM types are skipped. Two sub-patterns are generated:

- (a) **Cross-section sum (float)**. For each column c that appears in multiple types for the same entity, sum the entity’s values across all types: $\sum_i \mathcal{E}[t_i, \text{eid}_i]$ where each eid_i shares the same base entity and column. Requires the column to appear in ≥ 2 types.
- (b) **Cross-section difference (float)**. For each pair of types (t_1, t_2) sharing a common column c for the same entity, compute $\mathcal{E}[t_1, e/c] - \mathcal{E}[t_2, e/c]$.

Graph answer function. (a) Multiple `lookup` calls (one per type), then summation; (b) two `lookup` calls, then subtraction.

Answer type. float.

Sampling and Validation. Each generator produces a pool of all structurally valid candidates from the graph—the pool sizes range from 8 (multi-hop in Basel Capital, which has only 27 ENM entries) to 2,443 (cross-reference in Credit Portfolio, which has 2,334 triples). From each pool, up to N questions are sampled uniformly without replacement using a fixed random seed (default $N = 10$, seed 42) for reproducibility. Before inclusion, every candidate is *validated*: the graph answer function is re-executed and its output compared to the stored ground truth. Questions where the answer is `None`, an empty set, or inconsistent with the stored ground truth are discarded. This double-check guards against bugs in the answer function and ensures that the benchmark is self-consistent.

Table 2: Comparison of question generation approaches. ✓ = fully satisfied, ~ = partially satisfied, × = not satisfied.

Approach	Provable GT	Zero cost	Reproducible	No contam.	Ling. diversity
Human annotation	×	×	✓	✓	✓
LLM-driven	~	×	×	×	✓
Template + crowdsource	✓	×	~	✓	✓
Graph-topology (ours)	✓	✓	✓	✓	×
Hybrid (ours, optional)	✓	×	~	✓	✓

Ground Truth by Construction. A key property of FinStructBench is that ground truth is *defined* by graph traversal, not by human annotation. The ingester’s correctness can be verified independently: we confirm that every stored ENM value appears verbatim in the source markdown (1,572/1,572 entries verified across all five instances). Given correct ingestion, the answer function is guaranteed correct by the determinism of key lookup, pattern matching, and set operations. This eliminates annotation noise and enables scaling to arbitrarily large question sets.

3.3 Design Rationale: Deterministic vs. Hybrid Generation

The question generation methodologies surveyed in Section 2 can be organized along two axes: the *source of structural logic* (how the question’s answerable form is determined) and the *source of natural language* (how the question is expressed). Table 2 compares representative approaches along five properties critical for benchmark integrity.

FinStructBench’s default **deterministic mode** prioritizes the first four properties—provable ground truth, zero marginal cost, bit-exact reproducibility, and zero contamination risk—at the expense of linguistic diversity. We justify this tradeoff on three grounds:

1. **Ground-truth integrity.** In financial regulation, an incorrect ground-truth label can lead to false confidence in a model’s accuracy on safety-critical tasks. Human annotation introduces noise (FinQA reports inter-annotator agreement of 92.2% [Chen et al., 2021]), and LLM-generated labels require expensive verification. Deterministic graph traversal eliminates both sources of error.
2. **Contamination resistance.** LLM-generated benchmarks risk circular evaluation: the same model family that generated the questions may have memorized the patterns. Dynamic-KGQA [Dammu et al., 2025] addresses this with fresh splits, but cannot guarantee that the generating model did not influence the question distribution. Our rule-based generators are immune to this concern.
3. **Scalability.** Adding a new benchmark instance requires only a markdown document—no API calls, no annotation budget, and no crowdsourcing pipeline. The ten generators automatically produce a question pool whose size scales with the document’s graph complexity.

Optional Hybrid Mode. To address the linguistic diversity limitation, FinStructBench provides an optional **hybrid mode** inspired by the GrailQA pipeline [Gu et al., 2021]: the deterministic

generators produce the question logic, graph answer function, and ground truth as before, and an LLM is then used solely to paraphrase the template-generated natural language into more diverse, realistic phrasing. Crucially, the paraphrase step modifies only the surface form of the question—the ground truth, the graph answer function, and the LLM evaluation prompt remain unchanged, preserving provable correctness.

The toolkit exposes this choice via a `question_mode` parameter (default: "deterministic"). In hybrid mode, each question’s `natural_language` field is replaced by an LLM-generated paraphrase, while the `llm_prompt` (the structured evaluation prompt sent to the model under test) is left unchanged. This design ensures that:

- The LLM under test sees a fixed, format-controlled prompt regardless of mode—results remain comparable.
- The paraphrase LLM is used only for benchmark *presentation*, never for ground-truth *computation*.
- Users can reproduce the deterministic baseline at any time by switching modes, with no change to scoring or ground truth.

3.4 Evaluation Protocol

Each question is evaluated against two systems:

1. **Graph Baseline:** The answer function is executed directly on the DocumentGraph via deterministic traversal (key lookups, triple pattern matching, set operations). No learning or training is involved—the graph achieves 100% by construction and serves as the upper bound.
2. **LLM Under Test:** The LLM receives the full markdown document wrapped in `<report>` tags as context, along with a system prompt instructing it to answer precisely using only the provided data and to copy numbers exactly as they appear. Each question includes a format instruction specifying the expected answer tag (e.g., `ANSWER: 13.232`).

Response Parsing. LLM responses are parsed using type-specific parsers with two-stage extraction:

- **Float:** First attempts regex on `ANSWER:\s*<number>`; falls back to the last float literal in the response.
- **Int:** Same two-stage strategy; fallback extracts integers in the range `[1, 999]`.
- **Bool:** Matches `ANSWER:\s*(true|false|yes|no)`; falls back to keyword detection (`passes`, `does not`, `fails`).
- **Set:** Splits the `ANSWER:` line on commas and strips quotes.
- **String:** Returns the `ANSWER:` line verbatim.

Scoring. Scoring uses exact match with the following type-specific rules: float answers require $|a - g| < 10^{-6}$; integer answers require equality after casting; boolean answers require equality; set answers require exact set equality (superset answers are also accepted); string answers require case-insensitive equality. Tuple-valued answers (multi-hop) compare element-wise with float tolerance 10^{-3} .

Table 3: FinStructBench instance statistics.

Instance	ENM	Triples	Questions	Categories	Regulatory Context
Model Validation	275	5,586	100	10	SR 11-7
Fair Lending	1,139	2,014	90	9	ECOA / HMDA
Stress Test	476	1,417	90	9	CCAR / DFAST
Credit Portfolio	1,503	2,356	90	9	OCC Guidelines
Basel Capital	494	784	84	9	Basel III Pillar 3
Total	3,887	12,157	454	—	—

4 Benchmark Instances

We release five benchmark instances covering major financial document types encountered in US banking regulation. Each instance is a synthetic but structurally realistic report: we first defined the table schemas, section structure, and entity names to mirror actual regulatory filings, then populated numeric cells with randomly generated values drawn from domain-appropriate distributions (e.g., capital ratios from $\mathcal{N}(12, 3)$ clipped to $[4, 25]$, approval rates from $\text{Beta}(5, 2)$). All instances are generated with fixed random seeds for reproducibility. Table 3 summarizes their characteristics.

Model Validation (SR 11-7). A model risk management report evaluating a credit scoring model across multiple segments. Contains accuracy metrics (AUC, precision, recall, F1), feature importance analysis (FANOVA), slicing performance by demographic segments, and resolution analysis. This is the only instance with contradiction questions, as it contains per-feature pass/fail assessments that can be inconsistent across segments.

Fair Lending (ECOA/HMDA). A fair lending analysis examining loan disposition patterns across protected classes (race, gender, age). Contains adverse impact ratios, pricing analysis with statistical significance tests, geographic distribution indicators, and model fairness metrics (demographic parity, equalized odds). The largest instance by ENM entries (1,139), reflecting the high dimensionality of cross-tabulated demographic data.

Stress Test (CCAR/DFAST). A capital adequacy stress test with macroeconomic scenarios (baseline, adverse, severely adverse) projected across nine quarters. Contains loan loss projections by portfolio segment, pre-provision net revenue forecasts, capital ratio trajectories, and capital action plans.

Credit Portfolio Review (OCC). A credit portfolio examination covering portfolio composition, credit quality metrics, risk rating distributions, concentration analysis, allowance adequacy, vintage analysis, migration matrices and stress loss projections.

Basel Capital Adequacy (Pillar 3). A Basel III regulatory disclosure with capital composition, risk-weighted assets by exposure class, capital ratios, credit risk parameters (PD, LGD, EAD), market risk (VaR), operational risk (SMA) and leverage ratios. The smallest instance by ENM entries (494) but with complex hierarchical table structure.

Table 4: Overall benchmark results: Graph baseline vs. Claude Sonnet 4 in deterministic and hybrid question modes.

Instance	Questions	Graph	Det. LLM	Hyb. LLM	Δ
Model Validation	100	100 (100%)	69 (69%)	71 (71%)	+2
Fair Lending	90	90 (100%)	57 (63%)	59 (66%)	+2
Stress Test	90	90 (100%)	27 (30%)	27 (30%)	0
Credit Portfolio	90	90 (100%)	53 (59%)	57 (63%)	+4
Basel Capital	84	84 (100%)	57 (68%)	54 (64%)	-3
Total	454	454 (100%)	263 (58%)	268 (59%)	+5

5 Experiments

5.1 Setup

We evaluate Claude Sonnet 4 (`claude-sonnet-4-20250514`) against the graph baseline across all five instances and all ten question categories. The LLM receives the complete markdown document as context (500–1,000 lines) along with a structured prompt for each question specifying the expected answer format. We use 10 questions per category (where available), sampled with seed 42, yielding 454 questions per mode.

To assess the effect of question phrasing on LLM performance, we run the full benchmark in two modes: *deterministic* (template-generated questions) and *hybrid* (questions paraphrased by an LLM for linguistic diversity while preserving ground truth; see Section 3.2). The graph baseline achieves 100% accuracy in both modes by construction, confirming that paraphrasing does not alter the ground truth.

5.2 Main Results

Table 4 presents the overall results. The graph baseline achieves 100% across all instances by construction. Claude Sonnet 4 achieves 58% overall in deterministic mode and 59% in hybrid mode—a marginal improvement of 5 questions (+1 pp). Per-instance scores range from 30% (Stress Test) to 69–71% (Model Validation), reflecting substantial variation in document complexity.

5.3 Results by Question Category

Table 5 reveals a clear complexity gradient across the ten question categories. We highlight the key patterns:

- **Absence (94%):** Verifying that an entity does not exist in the graph. The LLM excels at this task because it requires confirming the non-existence of a specific value—a form of focused search that plays to the model’s strengths.
- **Threshold (74%):** Simple inequality checks against a single value. The LLM only needs to locate one number and compare it to a threshold.
- **Multi-Hop (68%):** Chained lookup requiring an intermediate computation (e.g., `argmin/argmax`) followed by a cross-type lookup. Performance varies substantially across instances (30–90%), suggesting sensitivity to table structure.

Table 5: Results by question category (aggregated across all instances), ordered by decreasing deterministic LLM accuracy.

Category	Graph Op.	Total	Graph	Det. LLM	Hyb. LLM	Δ
Absence	$\nexists(h, r, t)$	50	50	47 (94%)	47 (94%)	0
Threshold	$\phi(v) \geq \tau$	50	50	37 (74%)	36 (72%)	-1
Multi-Hop	$\text{chain}(f_1, f_2)$	50	50	34 (68%)	33 (66%)	-1
Ranking	$\text{sort}(\{v_i\})[k]$	50	50	33 (66%)	33 (66%)	0
Cross-Reference	$S_1 \cap S_2$	50	50	29 (58%)	34 (68%)	+5
Exact Recall	$\mathcal{E}[k]$	50	50	26 (52%)	24 (48%)	-2
Cross-Table Agg.	$\sum_i f(T_i)$	44	44	21 (48%)	21 (48%)	0
Numeric Comp.	$f(v_1, v_2)$	50	50	17 (34%)	18 (36%)	+1
Counting	$ \{h : (h, r, t)\} $	50	50	16 (32%)	19 (38%)	+3
Contradiction	$\exists \text{pass} \wedge \exists \text{fail}$	10	10	3 (30%)	3 (30%)	0
Total	—	454	454	263 (58%)	268 (59%)	+5

Table 6: Claude Sonnet 4 accuracy by instance and category (deterministic mode). Each cell shows correct/total. Shading indicates: ≥80%, 40–79%, <40%.

	Absence	Threshold	Multi-Hop	Ranking	Cross-Ref	Exact Recall	Cross-Tbl Agg	Numeric Comp	Counting	Contradiction
Model Valid.	9/10	10/10	8/10	10/10	7/10	6/10	10/10	3/10	3/10	3/10
Fair Lending	10/10	10/10	6/10	7/10	5/10	6/10	7/10	2/10	4/10	—
Stress Test	8/10	7/10	3/10	1/10	7/10	0/10	0/10	0/10	1/10	—
Credit Port.	10/10	4/10	8/10	6/10	7/10	9/10	0/10	6/10	3/10	—
Basel Capital	10/10	6/10	9/10	9/10	3/10	5/10	4/4	6/10	5/10	—

- **Ranking and Cross-Reference (58–66%):** Tasks requiring comparison across multiple entities or relation types. Cross-reference shows the largest hybrid improvement (+5), suggesting that more natural phrasing helps the LLM parse set-intersection queries.
- **Exact Recall (52%):** Precise numeric lookup. Despite being a single-hop operation, LLMs frequently return nearby values from the same table or confuse similarly-named entities.
- **Cross-Table Aggregation (48%):** Aggregating values across multiple tables. Requires identifying and combining information from different document sections.
- **Numeric Computation (34%) and Counting (32%):** Exhaustive enumeration or arithmetic over extracted values. LLMs tend to undercount or introduce calculation errors.
- **Contradiction (30%):** Global consistency checking across segments—the most challenging category (though limited to 10 questions from the Model Validation instance).

5.4 Per-Instance Breakdown

Table 6 shows the full instance $\times$ category breakdown in deterministic mode. Several patterns emerge:

Table 7: Deterministic vs. hybrid LLM accuracy by category (aggregated across all five instances, 454 questions per mode). The graph baseline achieves 100% in both modes.

Category	Det. LLM	Hyb. LLM	Δ
Absence	47/50 (94%)	47/50 (94%)	0
Contradiction	3/10 (30%)	3/10 (30%)	0
Counting	16/50 (32%)	19/50 (38%)	+3
Cross-Reference	29/50 (58%)	34/50 (68%)	+5
Cross-Table Agg.	21/44 (48%)	21/44 (48%)	0
Exact Recall	26/50 (52%)	24/50 (48%)	-2
Multi-Hop	34/50 (68%)	33/50 (66%)	-1
Numeric Comp.	17/50 (34%)	18/50 (36%)	+1
Ranking	33/50 (66%)	33/50 (66%)	0
Threshold	37/50 (74%)	36/50 (72%)	-1
Total	263/454 (58%)	268/454 (59%)	+5

Absence is consistently high. All five instances score $\geq 80\%$ on absence detection, confirming that LLMs are reliable at verifying that a specific value does not appear in the document.

Stress Test is consistently hard. Stress Test scores only 30% overall, with 0/10 on exact recall, numeric computation, and cross-table aggregation. The dense quarter $\times$ scenario matrices cause systematic extraction failures.

Instance difficulty varies widely. Model Validation is the easiest instance (69%), benefiting from well-labeled tables and distinctive entity names. Stress Test (30%) and Credit Portfolio (59%) are harder due to wide matrices and complex hierarchical structures.

Category difficulty is instance-dependent. Multi-hop ranges from 3/10 (Stress Test) to 9/10 (Basel Capital), and cross-table aggregation ranges from 0/10 (Stress Test, Credit Portfolio) to 10/10 (Model Validation), demonstrating that structural complexity interacts with document layout.

5.5 Deterministic vs. Hybrid Question Mode

Table 7 compares LLM performance across the two question modes. Hybrid paraphrasing yields a net gain of 5 questions (+1 pp overall), but the effect is category-dependent:

- **Cross-reference benefits most** (+5, from 58% to 68%). Template-generated cross-reference questions use rigid phrasing with nested quoting; the paraphrased versions use more natural set-intersection language that may better match the LLM’s training distribution.
- **Counting improves modestly** (+3, from 32% to 38%). Natural phrasing of enumeration queries may reduce misinterpretation of the counting target.
- **Exact recall slightly degrades** (-2, from 52% to 48%). Paraphrasing may remove disambiguating cues (e.g., quoted column names) that help the LLM locate the correct cell.
- **Most categories are unaffected** (6 of 10 categories show ≤ 1 question difference), indicating that the question surface form has limited impact on LLM accuracy for most task types.

Table 8: Example questions in deterministic vs. hybrid mode. The hybrid mode produces more natural phrasing while preserving the exact same meaning and all named entities.

Category	Deterministic (template)	Hybrid (paraphrased)
Exact Recall	What is the 'Value' value for 'Utilization' (category: fanova...)?	What is the Value for Utilization in the fanova_main_effect_curves category?
Threshold	Does 'test/AUC' (model.accuracy) meet better than random (AUC $\geq$ 0.5)?	Does the 'test/AUC' model.accuracy achieve performance better than random chance...?
Contradiction	Which features have contradictory performance across segments?	Which features exhibit contradictory performance when analyzed across different segments?

Table 8 illustrates the paraphrasing effect: template artifacts (extra quoting, rigid phrasing) are replaced with natural, professional language while preserving all entity names, column names, and threshold values exactly.

5.6 Error Analysis

We analyze the 191 LLM failures (deterministic mode) across representative error modes:

Adjacent Cell Confusion. In exact recall failures, the LLM frequently returns a value from a neighboring cell in the same table. For example, when asked for “Q1 2026/Non-Interest Income,” Claude returns the Non-Interest Expense value from the same row. This error is especially prevalent in the Stress Test instance (0/10 on exact recall), which uses wide matrices with 9 quarterly columns.

Partial Enumeration. In the majority of counting failures (34 of 50 questions missed), the LLM provides a count lower than the ground truth. The model appears to enumerate entities until it reaches a “satisfying” count rather than exhaustively scanning all triples.

Cross-Table Blindness. Cross-table aggregation achieves 0/10 on both Stress Test and Credit Portfolio, where the required values span multiple sections. The LLM typically finds values from one table but misses contributions from other sections of the document.

Arithmetic Errors. Numeric computation failures (33 of 50 missed) frequently involve correct value extraction but incorrect arithmetic—for example, computing a ratio with the numerator and denominator swapped, or rounding to the wrong precision.

5.7 Ingestion Mode Comparison

The preceding experiments use a deterministic regex-based ingestion pipeline to construct the ground-truth graph. A natural question is whether an LLM could replace or augment this pipeline. We compare three ingestion strategies across all five benchmark instances:

1. **Regex** (default): Pure deterministic extraction using regex pattern matching, string parsing and value-pattern heuristics.

Table 9: ENM and unique triple counts by ingestion mode across five instances. Triple counts are deduplicated (cf. Table 3, which reports raw triples including duplicates). The LLM-only mode recovers only 12.5% of regex ENM entries, while hybrid recovers all regex entries and adds prose-extracted data.

Instance	ENM Entries			Triples		
	Regex	LLM	Hybrid	Regex	LLM	Hybrid
Model Validation	275	77	1,107	2,109	159	3,041
Fair Lending	1,139	109	1,140	1,611	279	1,594
Stress Test	476	35	480	1,243	105	1,305
Credit Portfolio	1,503	150	1,520	2,256	422	2,340
Basel Capital	494	116	498	778	286	803
Total	3,887	487	4,745	7,997	1,251	9,083

Table 10: Pairwise Jaccard index and recovery rates. Regex $\leftrightarrow$ LLM overlap is near zero because the LLM uses different naming conventions. Hybrid preserves all regex values exactly (100% value match) while recovering 97.2% of regex ENM entries on average.

Instance	Regex $\leftrightarrow$ LLM		Regex $\leftrightarrow$ Hybrid		Recovery (%)	
	ENM J	Triple J	ENM J	Triple J	LLM	Hybrid
Model Validation	0.048	0.023	0.207	0.084	5.8	86.2
Fair Lending	0.000	0.043	0.999	0.895	0.0	100.0
Stress Test	0.008	0.003	0.992	0.952	0.8	100.0
Credit Portfolio	0.000	0.012	0.989	0.793	0.0	100.0
Basel Capital	0.020	0.057	0.992	0.813	2.4	100.0
Mean	0.015	0.028	0.836	0.707	1.8	97.2

2. **LLM-Only:** The LLM (Claude Opus 4) extracts all ENM entries and triples directly from each table and prose section, with no regex preprocessing.
3. **Hybrid:** Regex extraction as the base, with LLM fallback for ambiguous column classification, entity resolution and prose relation extraction.

Metrics. We measure: (i) ENM and triple counts per mode, (ii) pairwise Jaccard index over ENM key sets and triple sets, (iii) value match rate (exact numeric agreement on shared ENM keys), (iv) regex recovery rate (fraction of regex ENM entries recovered by each alternative mode), and (v) numeric precision (exact match and $< 10^{-6}$ tolerance on shared keys).

LLM-only extraction is catastrophically incomplete. Table 9 shows that the LLM-only mode extracts only 487 of 3,887 regex ENM entries (12.5%) and 1,251 of 7,997 triples (15.6%). Table 10 reveals that even the entries the LLM does extract have near-zero overlap with the regex graph: the mean ENM Jaccard index is 0.015 and the mean regex recovery rate is just 1.8%. This near-total disjointness arises because the LLM generates different entity identifiers (e.g., “CET1_Capital_Ratio” vs. the regex-produced “cet1_capital_ratio/capital_ratios”), different section tags and tends to summarize rather than exhaustively enumerate all table cells.

Hybrid mode is a strict superset of regex. The hybrid mode recovers 97.2% of regex ENM entries on average (100% on four of five instances), with a 100% value match rate—every shared numeric value is bit-identical. The hybrid graph is strictly larger: 4,745 ENM entries (+22% over regex) and 9,083 triples (+14%), reflecting the additional prose-extracted data contributed by the LLM component. The one outlier is Model Validation (86.2% recovery, ENM Jaccard 0.207), where the LLM’s column classification differs from the heuristic on several ambiguous tables, leading to different entity-id construction for some entries.

Numeric precision is exact. On all shared ENM keys between regex and hybrid modes, 100% of numeric values match exactly (within floating-point representation). The hybrid mode introduces zero numeric errors across all 3,887 shared entries, confirming that the SHA-256 integrity-checked storage preserves values regardless of ingestion mode.

Implications. These results provide empirical justification for the regex-first architecture: the deterministic pipeline is not merely a design preference but a quantitative necessity. An LLM-only approach loses 87% of structured data and produces a graph with fundamentally different structure. The hybrid mode demonstrates that the optimal architecture uses regex as the backbone for exhaustive, precise extraction, with the LLM serving as an augmentation layer for content that falls outside the regex pipeline’s scope.

6 Discussion

Implications for Financial AI. The 41–42 percentage-point gap between graph-based retrieval (100%) and LLM performance (58–59%) has direct implications for the deployment of AI in financial regulation. While the LLM performs well on absence detection (94%) and threshold checking (74%), accuracy drops sharply on tasks requiring exhaustive enumeration, arithmetic or cross-table reasoning (30–48%). These findings suggest that LLM-only approaches are insufficient for reliable financial document analysis. Hybrid architectures that combine LLM understanding with structured graph retrieval are likely necessary for production-grade deployment.

Why Rule-Based Ingestion? FinStructBench’s ingestion pipeline is deliberately rule-based (regex pattern matching, string parsing, value-pattern heuristics) rather than using the SOTA extraction techniques surveyed in Section 2. The ingestion mode comparison in Section 5.7 provides direct empirical evidence: an LLM-only pipeline recovers only 12.5% of structured data with near-zero overlap (Jaccard = 0.015). Beyond this empirical finding, the choice is driven by three requirements specific to benchmark construction:

1. **Provable correctness.** The ground truth must be verifiably correct by construction. LLM-based extraction (e.g., GPT-4 or Claude applied to table parsing) achieves 68–74% on FinQA [Chen et al., 2021]—accurate enough for many applications, but insufficient as a ground-truth oracle. Any extraction error would propagate into incorrect benchmark answers, undermining the benchmark’s purpose. Rule-based parsing achieves near-100% precision on well-structured markdown tables because the format is unambiguous.
2. **Determinism and reproducibility.** Given the same document, the pipeline must produce the identical graph on every run. LLM outputs are inherently stochastic (even at temperature 0, sampling artifacts can vary across API versions), making them unsuitable for a reproducible ground-truth engine. Rule-based parsing is fully deterministic.

3. **Transparency and auditability.** In a regulated financial setting, every ground-truth answer must be traceable to a specific cell in the source document. The SHA-256 integrity hash on each ENM entry provides a cryptographic audit trail from answer to source value. LLM-based extraction is opaque—there is no equivalent mechanism to verify that an extracted value matches the source.

The tradeoff is generalization: the rule-based pipeline requires well-structured markdown with pipe-delimited tables. It does not handle prose-embedded numbers, scanned PDFs or irregular layouts that layout-aware models like LayoutLMv3 [Huang et al., 2022] or Donut [Kim et al., 2022] can process. For real-world deployment beyond the benchmark, we provide a **hybrid ingestion mode** (described below) that uses an LLM to assist with column classification, entity resolution and relation extraction, while preserving the deterministic ENM storage and integrity hashing.

Hybrid Ingestion Architecture. To bridge the gap between rule-based precision and LLM-based flexibility, the toolkit supports a hybrid ingestion mode (`--ingest-mode hybrid`) in which the LLM augments three specific steps of the pipeline:

- **Column classification:** When heuristic classification is ambiguous (e.g., mixed columns, domain-specific labels), the LLM is prompted to classify each column as entity, numeric, boolean or label, given the table headers and a sample of rows.
- **Entity resolution:** When the entity column cannot be detected by name heuristic, the LLM identifies which column contains the primary entity identifier.
- **Relation extraction:** For non-tabular content (prose paragraphs, nested lists), the LLM extracts (entity, relation, value) triples that the regex pipeline would miss.

Critically, the hybrid mode preserves the core invariant: all numeric values are stored with SHA-256 hashes and verified on lookup. The LLM assists with *structural* decisions (which column is the entity? what relation does this paragraph express?) but the *values* remain deterministically extracted and integrity-checked. This design ensures that even in hybrid mode, the ground truth is verifiable against the source document. Users select the ingestion mode via the CLI (`--ingest-mode default|hybrid`) or the Python API (`ingest.markdown(path, mode="hybrid")`).

Benchmark as a Development Tool. FinStructBench can serve as an evaluation harness during the development of retrieval-augmented generation (RAG) systems, agentic frameworks and tool-use architectures for financial documents. The provably correct ground truth eliminates the need for human evaluation, enabling rapid iteration on system design.

Extensibility. The benchmark toolkit is designed for extensibility. New question categories can be added by implementing a generator class with a `generate(graph)` method. New instances can be created from any markdown document. The phase encoder system can be extended with domain-specific threshold rules.

Limitations. FinStructBench currently uses synthetic financial documents rather than real regulatory filings. While the synthetic reports are designed to be structurally realistic, real documents may contain additional complexity (e.g., footnotes, cross-references to external regulations, redacted sections). We plan to extend the benchmark with anonymized real-world instances in future work.

The benchmark evaluates a single LLM (Claude Sonnet 4) in a zero-shot, full-context setting. While we compare deterministic and hybrid question modes, both use the same underlying ground truth. Future work should evaluate multiple models, few-shot prompting, chain-of-thought reasoning and tool-augmented architectures.

7 Released Artifacts

FinStructBench is released as an open-source Python package with the following components:

1. **Toolkit** (`fstructbench/`): Document ingester, ten question generators, scoring module, LLM caller and benchmark orchestrator.
2. **Instances** (`instances/`): Five markdown financial reports with reproducible generator scripts.
3. **Results** (`results/`): Baseline results with per-question details in JSON format.
4. **CLI**: `python -m fstructbench run <doc.md>` with options for category limit, model and graph-only mode.

8 Conclusion

We presented FinStructBench, a benchmark for structured information retrieval from financial documents with provably correct ground truth. Three sets of experiments yield complementary findings:

1. **LLM evaluation.** Across 454 questions spanning ten categories and five financial document types, Claude Sonnet 4 achieves 58–59% overall against the 100% graph baseline, with a clear complexity gradient: 94% on absence detection but only 30% on contradiction and 32% on counting.
2. **Question mode comparison.** Comparing deterministic (template-based) and hybrid (LLM-paraphrased) question modes reveals a marginal +1 pp improvement overall, with cross-reference queries benefiting most (+10 pp).
3. **Ingestion mode comparison.** A three-way comparison of regex-only, LLM-only and hybrid ingestion using Claude Opus 4 demonstrates that LLM-only extraction recovers only 12.5% of structured data (Jaccard = 0.015), while hybrid ingestion preserves 100% of regex values exactly and adds 22% more entries from prose extraction.

Together these results demonstrate that current LLMs are insufficient for reliable financial document analysis: they struggle with structured retrieval tasks (58% accuracy) and cannot replace deterministic extraction pipelines (12.5% recovery). Hybrid architectures that combine rule-based precision with LLM flexibility offer the most promising path forward. FinStructBench provides a rigorous, extensible evaluation framework for measuring progress toward this goal.

References

Alvarado, J. C. S., Verspoor, K., and Baldwin, T. (2015). Domain adaption of named entity recognition to support credit risk assessment. In *Proceedings of the Australasian Language Technology Association Workshop*, pp. 84–90.

- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., and Wang, W. Y. (2020). HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of EMNLP*, pp. 1026–1036.
- Chen, Z., Chen, W., Smiley, C., Shah, S., Borber, I., Leu, S., Zhu, Y., Kumar, S., Zhang, J., and Wang, W. Y. (2021). FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*, pp. 7862–7872.
- Chen, Z., Li, S., Smiley, C., Ma, Z., Shah, S., and Wang, W. Y. (2022). ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of EMNLP*, pp. 6279–6292.
- Herzig, J., Nowak, P. K., Müller, T., Piccinno, F., and Eisenschlos, J. (2020). TAPAS: Weakly supervised table parsing via pre-training. In *Proceedings of ACL*, pp. 4320–4333.
- Huang, Y., Lv, T., Cui, L., Lu, Y., and Wei, F. (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*.
- Iyyer, M., Yih, W., and Chang, M. (2017). Search-based neural structured learning for sequential question answering. In *Proceedings of ACL*, pp. 1821–1831.
- Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., and Wen, J.-R. (2023). StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of EMNLP*, pp. 9237–9251.
- Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., and Park, S. (2022). OCR-free document understanding transformer. In *European Conference on Computer Vision (ECCV)*.
- Lee, K., Joshi, M., Turc, I., Hu, H., Liu, F., Eisenschlos, J., Khandelwal, U., Shaw, P., Chang, M.-W., and Toutanova, K. (2023). Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of ICML*, pp. 18893–18912.
- Li, Y., Packer, C., and Gonzalez, J. E. (2024). How long can open-source LLMs truly promise on context length? *arXiv preprint arXiv:2402.17688*.
- Liu, Q., Chen, B., Guo, J., Ziyadi, M., Lin, Z., Chen, W., and Lou, J.-G. (2022). TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*.
- Malo, P., Sinha, A., Korhonen, P., Wallenius, J., and Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4):782–796.
- Mathew, M., Karatzas, D., and Jawahar, C. V. (2021). DocVQA: A dataset for VQA on document images. In *Proceedings of WACV*, pp. 2200–2209.

- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Pasupat, P. and Liang, P. (2015). Compositional semantic parsing on semi-structured tables. In *Proceedings of ACL*, pp. 1470–1480.
- Wang, Z., Zhang, H., Li, C.-L., Eisenschlos, J. M., Perot, V., Wang, Z., Miculicich, L., Fujii, Y., Shang, J., Lee, C.-Y., and Pfister, T. (2024). Chain-of-Table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations (ICLR)*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- Ye, Y., Hui, B., Yang, M., Li, B., Huang, F., and Li, Y. (2023). Large language models are versatile decomposers: Decompose evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 174–184.
- Shah, R., Chawla, K., Eidnani, D., Shah, A., Du, W., Chava, S., Raman, N., Smiley, C., Chen, J., and Yang, D. (2022). When FLUE meets FLANG: Benchmarks and large pre-trained language model for financial domain. In *Proceedings of EMNLP*, pp. 2322–2335.
- Zhang, J., Chen, B., Zhang, L., Ke, X., and Ding, H. (2022). Neural, symbolic and neural-symbolic reasoning on knowledge graphs. *AI Open*, 3:109–124.
- Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., and Chua, T. (2021). TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of ACL*, pp. 3277–3287.
- Butt, N., Chandrasekaran, V., Joshi, N., Nushi, B., and Balachandran, V. (2024). BenchAgents: Automated benchmark creation with multi-agent systems. *arXiv preprint arXiv:2410.22584*.
- Dammu, P. P. S., Naidu, H., and Shah, C. (2025). Dynamic-KGQA: A scalable framework for generating adaptive question answering datasets from knowledge graphs. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- Gu, Y., Kase, S., Vanni, M., Sadler, B., Liang, P., Yan, X., and Su, Y. (2021). Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases. In *Proceedings of The Web Conference (WWW)*, pp. 3477–3488.
- Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., and Vidgen, B. (2023). FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*.
- Matlin, G., Theerthala, A., Gupta, A., JM, A., Castilla, R., Ng, Y. M., and Chava, S. (2026). FinForge: Semi-synthetic financial benchmark generation. *arXiv preprint arXiv:2601.06747*.
- Shashidhar, S., Fourrier, C., Lozovskia, A., Wolf, T., Tur, G., and Hakkani-Tur, D. (2025). YourBench: Easy custom evaluation sets for everyone. In *Proceedings of the Conference on Language Modeling (COLM)*.

- Tang, J., Zhang, Q., Li, Y., Chen, N., and Li, J. (2025). GraphArena: Evaluating and exploring large language models on graph computation. In *International Conference on Learning Representations (ICLR)*.
- Tatarinov, A., Kannan, A., Srinivasa, J., Raj, B., Anand, A., Singh, A., Luthra, N., Lade, P., Shah, R., and Chava, S. (2025). KG-QAGen: A knowledge-graph-based framework for systematic question generation and long-context LLM evaluation. *arXiv preprint arXiv:2505.12495*.
- Trivedi, P., Maheshwari, G., Dubey, M., and Lehmann, J. (2017). LC-QuAD: A corpus for complex question answering over knowledge graphs. In *International Semantic Web Conference (ISWC)*, LNCS vol. 10588, pp. 210–218. Springer.
- Wu, Y., Tang, J., and others (2025). TableBench: A comprehensive and complex benchmark for table question answering. In *Proceedings of AAAI*, vol. 39, pp. 25497–25506.
- Zhu, J., Wang, J., Yu, B., Wu, X., Li, J., Wang, L., and Xu, N. (2025). TableEval: A real-world benchmark for complex, multilingual, and multi-structured table question answering. In *Proceedings of EMNLP*.